

01-SI — How can I inspect a Python function, variable by variable, in Spyder? / 01-SI — 如何在 Spyder 中逐个理解 Python 函数里的变量?

Question / 问题 (Question)

I am new to Python. Given a function, how can I break it down in Spyder, inspect each variable's type and size, and understand what happens step by step?

作为 Python 新手，如果有一个函数 (function)，想查看其中每个变量 (variable) 的种类和大小 (size)，应该如何在 Spyder 中拆解这个函数 (break down a function)，逐步执行并理解各个变量？

The worked case is your pair of functions, `generate_data_local` and `generate_data_global`. **Pause inside the function, inspect its variables, then advance one line at a time.**

本篇使用你提供的 `generate_data_local` 和 `generate_data_global`。先暂停在函数内部，查看变量，再逐行向前执行。

1. Understand the data model / 1. 先理解数据模型 (Data model)

Both functions generate `N_TRAIN` input/output pairs for $y = \exp(-x / 2) + \text{noise}$. Here `N_TRAIN = 10`, and `NOISE_STD = 4 * 0.03 = 0.12`. Inputs are drawn uniformly between 0 and 6. The Gaussian noise has mean 0 and standard deviation 0.12; its variance is 0.0144. A particular set of ten noise samples need not have sample mean 0 or sample standard deviation 0.12. See [NumPy: normal distribution parameters](#) ^[1].

两个函数都生成 `N_TRAIN` 组输入 / 输出对 (input/output pairs)，模型是 $y = \exp(-x / 2) + \text{noise}$ 。本例 `N_TRAIN = 10`，`NOISE_STD = 4 * 0.03 = 0.12`。输入从 0 到 6 之间的均匀分布 (uniform distribution) 抽取；高斯噪声 (Gaussian noise) 的均值 (mean) 为 0，标准差 (standard deviation) 为 0.12，方差 (variance) 为 0.0144。一次抽取的 10 个噪声值，其样本均值和样本标准差不一定恰好等于这些参数。参见 [NumPy: 正态分布参数](#) ^[1]。

We expose `y_true` and `noise` as named intermediate variables so that the original expression can be inspected in three steps:

为了查看中间变量 (intermediate variables)，把原来的一行 `y = ...` 展开成三步：

```
y_true = np.exp(-x / 2.0)
noise = rng.normal(0.0, NOISE_STD, size=(N_TRAIN, 1))
y = y_true + noise
```

This preserves the original calculation and order of random draws. Both returned arrays have shape `(10, 1)`.

这三步保留原来的计算方式及随机抽样顺序。返回的两个数组 (arrays) 形状 (shape) 均为 `(10, 1)`。

2. Open and run the example / 2. 打开并运行示例 (Run the example)

Open `01_spyder_function_inspection.py` in Spyder. Its Python environment needs NumPy; no input files are needed.

在 Spyder 中打开 `01_spyder_function_inspection.py`。所用 Python 环境需要有 NumPy，不需要输入文件。

```

"""Inspect local/global random data generation in Spyder. Requires NumPy."""

import numpy as np

def generate_data_local(N_TRAIN, NOISE_STD):
    """Return x and noisy exp(-x/2), each shaped (N_TRAIN, 1).

    N_TRAIN is a positive integer; NOISE_STD is a nonnegative scalar.
    A fresh Generator seeded with 5 is created on every call.
    """
    rng = np.random.default_rng(5) # BREAKPOINT L1: before creating the local RNG
    x = rng.uniform(0.0, 6.0, size=(N_TRAIN, 1)) # BREAKPOINT L2: inspect rng
    y_true = np.exp(-x / 2.0)
    noise = rng.normal(0.0, NOISE_STD, size=(N_TRAIN, 1))
    y = y_true + noise
    return x, y # BREAKPOINT L3: inspect x, y_true, noise, and y

```

```

def generate_data_global(N_TRAIN, NOISE_STD):
    """Generate the same model using NumPy's legacy global RandomState.

    This demonstration resets and advances the global random state.
    """
    np.random.seed(5) # BREAKPOINT G1: before resetting global state
    x = np.random.uniform(0.0, 6.0, size=(N_TRAIN, 1))
    y_true = np.exp(-x / 2.0)
    noise = np.random.normal(0.0, NOISE_STD, size=(N_TRAIN, 1))
    y = y_true + noise
    return x, y # BREAKPOINT G2: inspect the global-RNG version's local variables

```

```

if __name__ == "__main__":
    # Distinct caller names keep both results available in Variable Explorer.
    x_local, y_local = generate_data_local(10, 4 * 0.03)
    print("Local Generator")
    print("x^T =", x_local.T) # (1, 10); x_local itself remains (10, 1)
    print("y^T =", y_local.T)

    x_global, y_global = generate_data_global(10, 4 * 0.03)
    print("Global RandomState")
    print("x^T =", x_global.T) # (1, 10); x_global itself remains (10, 1)
    print("y^T =", y_global.T)
    print("Same x arrays:", np.array_equal(x_local, x_global))

```

From a terminal at the repository root, you can also run:

也可以在仓库根目录的终端（terminal）中运行：

```
python examples/01_spyder_function_inspection.py
```

The script uses `x_local`, `y_local`, `x_global`, and `y_global` in the caller so both results remain available. Reusing the caller names `x`, `y` for the second call, as in the original snippet, would replace their references to the first result.

调用方（caller）使用 `x_local`、`y_local`、`x_global`、`y_global` 四个名称，方便同时查看两组结果。原代码两次都用 `x`、`y` 接收结果，第二次赋值会让这两个名称改为指向第二组数组。

Defining a function with `def` does not run its body; the calls in the last block do. Names assigned inside a function are normally local to that call. Importantly, **x and y are local in both functions**. The word `global` in the second function's name refers to the random-number state it uses, not the scope of `x` and `y`. See [Python: function definitions and local names](#) ^[2].

`def` 只定义函数（function definition），文件末尾的调用（function call）才执行函数体（function body）。函数内赋值的名称通常属于本次调用的局部变量（local variables）。**两个函数内部的 `x`、`y` 都是局部变量**。第二个函数名称中的 `global` 指它使用的随机数状态（random state），并不代表 `x`、`y` 是全局变量。参见 [Python：函数与局部名称](#) ^[2]。

3. Pause inside the local function / 3. 在局部生成器函数中暂停（Breakpoint）

1. Show **Variable Explorer** and **IPython Console** from **View > Panes**. See [Spyder: interface customization](#) ^[3].

通过 **View > Panes** 显示变量浏览器（**Variable Explorer**）和交互式控制台（**IPython Console**）。参见 [Spyder：界面设置](#) ^[3]。

2. Put the cursor on the executable line marked `BREAKPOINT L1` and choose **Debug > Toggle breakpoint**. Also set a breakpoint on the return line marked `BREAKPOINT L3`.

光标放在标有 `BREAKPOINT L1` 的可执行语句上，选择 **Debug > Toggle breakpoint** 设置断点（breakpoint）。再在标有 `BREAKPOINT L3` 的 `return` 行设置一个断点。

3. Start **Debug > Debug file**. If it pauses near the file's beginning, use **Continue** to reach `L1`.

选择 **Debug > Debug file**。如果先停在文件开头，用继续执行（**Continue**）到达 `L1`。

4. At `L1`, inspect `N_TRAIN` and `NOISE_STD`. The assignment to `rng` has not executed. Use **Step over / Next** once to create it, then inspect `rng`.

在 `L1`，先看 `N_TRAIN` 和 `NOISE_STD`。`rng` 的赋值语句（assignment）还没有执行。执行一次单步跳过（**Step over / Next**），再查看 `rng`。

5. Step through `x`, `y_true`, `noise`, and `y`. At `L3`, all these variables exist and `return` has not executed yet.

依次执行 `x`、`y_true`、`noise`、`y` 的赋值。在 `L3`，全部中间变量都已存在，`return` 尚未执行。

These instructions follow Spyder 6 documentation; your menu labels or shortcuts may differ. In the Debugger pane, select the `generate_data_local` stack frame if you previously selected another frame. Spyder can inspect variables in the paused frame. See [Spyder: Debugger](#) ^[4].

以上按 Spyder 6 文档整理，菜单与快捷键可能因版本而异。如果在调试器面板（**Debugger**）选过其他调用栈帧（stack frame），请选回 `generate_data_local`。Spyder 可以查看当前暂停栈帧中的变量。参见 [Spyder：调试器](#) ^[4]。

Variable Explorer displays **Name, Type, Size, Value**. Double-click `x` or `y` to inspect the ten rows. Its Size column summarizes different kinds of objects differently. If the pane filters out `rng`, inspect it in the console. See [Spyder: Variable Explorer](#) ^[5].

变量浏览器显示名称（**Name**）、类型（**Type**）、大小（**Size**）和值（**Value**）。双击 `x` 或 `y` 可以查看 10 行数据。Size 列对不同对象显示的是不同摘要；如果 `rng` 被过滤，可以在控制台中查看。参见 [Spyder: 变量浏览器](#) ^[5]。

Paused before this line 暂停在这一行执行之前 (before execution)	Newly available from the preceding assignment 前一步已经产生的变量
<code>rng = ...</code>	<code>N_TRAIN</code> , <code>NOISE_STD</code> are the inputs; <code>rng</code> does not exist yet 输入参数 <code>N_TRAIN</code> 、 <code>NOISE_STD</code> 已有； <code>rng</code> 尚未产生
<code>x = ...</code>	<code>rng</code>
<code>y_true = ...</code>	<code>x</code>
<code>noise = ...</code>	<code>y_true</code>
<code>y = ...</code>	<code>noise</code>
<code>return x, y</code>	<code>y</code> ; all intermediate variables can now be inspected <code>y</code> ；可以查看全部中间变量

4. Choose the stepping action / 4. 选择单步操作 (Stepping)

Action 操作	Debugger command 调试命令 (debugger command)	Purpose 用途
Step over / Next 单步跳过 (Step over / Next)	<code>next</code>	Execute the current line without following ordinary nested calls line by line. Breakpoints can still interrupt it. 执行当前行，不逐行跟进普通的内部调用；途中仍可能被断点打断。
Step into 单步进入 (Step into)	<code>step</code>	Enter a called Python function whose implementation you want to inspect. 进入想理解其实现的 Python 函数。
Step out / Return 跳出函数 (Step out / Return)	<code>return</code>	Run until the current function is about to return, then advance to its caller. 运行到当前函数即将返回，再向前执行即可回到调用方。
Continue 继续执行 (Continue)	<code>continue</code>	Resume until another breakpoint or execution finishes. 运行到下一个断点或执行结束。

Use the toolbar or the debugger prompt, often shown as `IPdb`. See [Python: debugger commands](#) ^[6].

使用工具栏按钮，或在通常显示为 `IPdb` 的调试提示符 (debugger prompt) 后输入命令。参见 [Python: 调试命令](#) ^[6]。

Use **Next** for the NumPy calls in this example. To practice **Step into**, pause on `x_local`, `y_local = generate_data_local(...)` and enter that call. To inspect the second function, set G1 and G2 and continue to it.

本例的 NumPy 调用用 **Next** 即可。想练习 **Step into**，就在 `x_local`, `y_local = generate_data_local(...)` 这一调用行暂停，再进入函数。想观察第二个函数，可设置 G1、G2 断点并继续运行。

5. Inspect type and size explicitly / 5. 明确查看类型与大小 (Type and size)

At L3, try these expressions in the debugging console:

在 L3 的调试控制台逐个输入：

Expression 表达式 (expression)	Result in this example 本例结果	Meaning 含义
type(N_TRAIN)	<class 'int'>	Integer parameter, value 10 整数 (integer) 参数, 值为 10
type(NOISE_STD)	<class 'float'>	Floating-point parameter, value 0.12 浮点数 (float) 参数, 值为 0.12
type(rng).__name__	'Generator'	Random-number generator object 随机数生成器对象 (random-number generator)
type(rng.bit_generator).__name__	'PCG64'	Its underlying bit generator in the tested version 测试版本所用的底层位生成器 (bit generator)
type(x)	<class 'numpy.ndarray'>	Python object type Python 对象类型 (object type)
x.dtype	dtype('float64')	Array element type 数组元素类型 (element type)
x.shape	(10, 1)	10 rows, 1 column 形状 (shape) : 10 行、1 列
x.ndim	2	Number of axes 轴数 / 维度数 (number of axes / dimensions)
x.size	10	Total number of elements 元素总数 (total number of elements)
len(x)	10	Length of its first axis 第一条轴的长度 (length of the first axis)
x.nbytes	80	Element storage: 10 × 8 bytes 元素存储字节数 (bytes) : 10 × 8
x.T.shape	(1, 10)	Shape of the transpose 转置 (transpose) 后的形状
len(x.T)	1	First-axis length of the transpose 转置后的第一条轴长度

nbytes excludes object overhead. type(x) and x.dtype answer different questions. See [NumPy: array attributes](#)^[7]. rng, a Python integer, and a Python float do not have an array .shape or .size; inspect their type and value instead.

nbytes 不包含对象的管理开销（object overhead）。type(x) 与 x.dtype 分别描述对象和元素的类型。参见 NumPy: 数组属性^[7]。rng、Python 整数和浮点数都没有数组的 .shape 或 .size，应查看它们的类型和值。

print(x.T) displays the transpose; it does not assign a new shape to x. (10, 1) is a 2D column, (1, 10) a 2D row, and (10,) a 1D array. A 1D array's .T remains 1D. In this example, mixing y_true of shape (10, 1) with noise of shape (10,) would broadcast to (10, 10), rather than produce ten paired observations. See NumPy: broadcasting^[8].

print(x.T) 只是显示转置结果，并没有给 x 重新赋值。(10, 1) 是二维列向量（column vector），(1, 10) 是二维行向量（row vector），(10,) 是一维数组（1D array）；一维数组转置后仍是一维。假如把本例 (10, 1) 的 y_true 与 (10,) 的噪声相加，广播（broadcasting）会生成 (10, 10) 的数组，无法表示原来所需的 10 对数据。参见 NumPy: 广播机制^[8]。

6. Give each variable a meaning / 6. 给每个变量一个含义 (Variable walkthrough)

Variable 变量 (variable)	Type 类型 (type)	Shape / size 形状 / 大小 (shape / size)	Role 含义 (role)
N_TRAIN	int	Scalar value 10 标量 (scalar)，值 10	Number of input/output pairs 输入 / 输出对数
NOISE_STD	float	Scalar value 0.12 标量，值 0.12	Noise standard deviation 噪声标准差
rng	Generator	No array shape 没有数组形状	Local random state; present only in generate_data_local 局部随机数状态；仅在 generate_data_local 中存在
x	ndarray, float64 ndarray, float64	(10, 1); 10 elements (10, 1); 10 个元素	Input locations 输入位置 (input locations)
y_true	ndarray, float64 ndarray, float64	(10, 1); 10 elements (10, 1); 10 个元素	Noise-free exp(-x / 2) 无噪声信号 (noise-free signal) exp(-x / 2)
noise	ndarray, float64 ndarray, float64	(10, 1); 10 elements (10, 1); 10 个元素	The actual noise samples used 实际加入的噪声样本 (noise samples)
y	ndarray, float64 ndarray, float64	(10, 1); 10 elements (10, 1); 10 个元素	Observed response, y_true + noise 观测响应 (observed response) y_true + noise

return x, y returns a two-item tuple. The caller unpacks it into two names; it does not form a single (10, 2) array. The intermediate names rng, y_true, and noise do not automatically become caller variables.

return x, y 返回一个包含两项的元组（tuple）。调用方解包（unpacking）后得到两个名称，并没有拼成一个 (10, 2) 数组。中间变量名称 rng、y_true、noise 也不会自动成为调用方的变量。

For a quick numeric check, the first three entries, rounded to six decimal places, are:

下面是测试运行中各数组前 3 个元素，保留 6 位小数（rounded values），可以用来快速对照：

Array 数组 (array)	First three entries 前 3 个元素
x_local	4.830018, 4.847645, 3.091953
y_local	0.122099, -0.059417, 0.098112
x_global	1.331959, 5.224394, 1.240315
y_global	0.536282, 0.033789, 0.394728

Negative observations are possible because the added Gaussian noise is not restricted to positive values.

观测值可以为负，因为加入的高斯噪声并不只取正值。

7. Compare local and global randomness / 7. 比较局部与全局随机数 (Local vs. global randomness)

Property 对照项	generate_data_local	generate_data_global
Random-number API 随机数接口 (API)	Separate Generator from default_rng(5) default_rng(5) 创建独立对象 Generator	NumPy's shared legacy RandomState 使用共享的传统 RandomState
Bit generator here 本例底层生成器 (bit generator)	PCG64	MT19937
Effect on the legacy global random state 对传统全局随机状态的影响	Does not reset or advance it 不重置、不推进该状态	Resets it to seed 5, then advances it 重置为种子 5，再通过抽样推进
Two calls with identical arguments 相同参数连续调用两次	Repeat the same data in the tested environment 在测试环境中重复同一组数据	Repeat the same data in the tested environment 在测试环境中重复同一组数据

See NumPy: Generator^[9], legacy random generation^[10], and np.random.seed^[11].

参见 NumPy: Generator^[9]、传统随机数接口^[10] 和 np.random.seed^[11]。

The same seed integer does not give these two APIs the same sequence. Both functions restart their respective generators inside every call, so repeated calls are not fresh independent datasets. Here, “local” means separate state, not new randomness each time. For successive datasets, create one generator outside the function and pass it in. Record NumPy’s version when exact reproduction matters; Generator streams are not guaranteed to remain identical across versions.

同一个种子（seed）整数，并不意味着这两套接口生成相同序列。两个函数都在每次调用内部重新初始化，因此反复调用得到的不是新的独立数据集。“局部”指状态独立管理，并不表示每次调用都产生新的随机结果。需要连续生成新数据时，可以在函数外创建一个生成器，再作为参数传入。精确复现时应记录 NumPy 版本；Generator 不保证跨版本的随机流（random stream）始终一致。

During debugging, inspect the stored noise variable. Calling `rng.normal(...)` again draws new noise and advances the state. Re-executing selected random-draw lines can therefore change the subsequent result.

调试时直接查看保存好的 `noise` 变量。再次输入 `rng.normal(...)` 会重新抽样并推进随机状态。 反复执行某一条随机抽样语句，可能改变后续结果。

8. Optional: inspect a scratch script line by line / 8. 可选：拆到临时脚本逐行运行 (Scratch script)

```
# %% Inputs
import numpy as np
N_TRAIN = 10
NOISE_STD = 4 * 0.03

# %% Calculation
rng = np.random.default_rng(5)
x = rng.uniform(0.0, 6.0, size=(N_TRAIN, 1))
y_true = np.exp(-x / 2.0)
noise = rng.normal(0.0, NOISE_STD, size=(N_TRAIN, 1))
y = y_true + noise
```

Execute the inputs first, then use **Run selection or current line** (default F9) for each statement. **Run cell** (Windows default Ctrl+Enter) executes a # %% block. See [Spyder: running code](#)^[12]. This top-level version gives the console its own variables. Running an isolated line from a function does not automatically supply its parameters; return cannot be run at the top level.

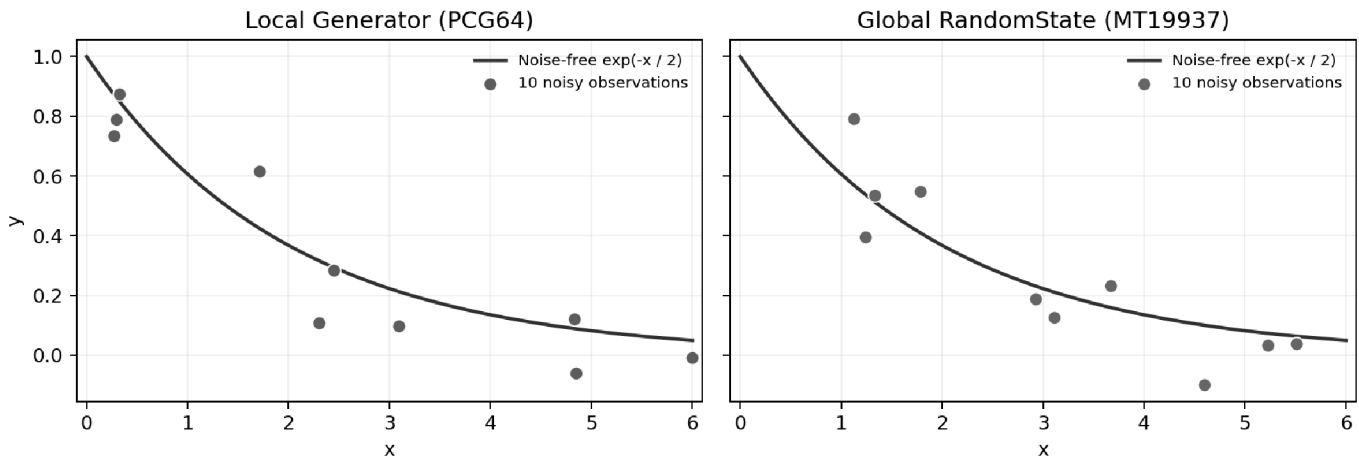
先执行输入部分，再用运行选中代码或当前行（**Run selection or current line**，默认 F9）逐句运行。运行代码单元（**Run cell**，Windows 默认 Ctrl+Enter）执行 # %% 划分的代码块。参见 [Spyder：运行代码](#)^[12]。这个顶层版本（top-level version）会在控制台中建立变量；只把函数内部一行送到普通控制台，不会自动补齐输入参数，return 也不能放在顶层执行。

If a variable is missing, check whether its assignment has run, whether the function already returned, and whether the correct console/frame and display filters are selected.

看不到变量时，检查赋值是否已执行、函数是否已返回，以及当前控制台、栈帧和过滤选项（filters）是否正确。

9. View the generated data / 9. 查看生成的数据 (Generated data)

Same seed = 5; different random streams



Noise standard deviation = 0.12; arrays x and y have shape (10, 1)

Both panels use 10 observations, seed 5, and noise standard deviation 0.12. Different random streams produce different points.

两幅图都使用 10 个观测值、种子 5 和标准差 0.12；不同随机流产生不同的数据点。

The figure is generated by `02_plot_generated_data.py`. For image insertion, see [Q2: images in Markdown](#).

图片由 `02_plot_generated_data.py` 生成。插图方法见问题 2：Markdown 中插入图片。

Verification: outputs, array shapes, equivalence to the original combined expressions, repeat calls, and global-state effects were checked using Python 3.9.25 and NumPy 1.26.4. Spyder instructions were checked against official documentation; the GUI sequence was not executed during preparation.

验证记录 (verification)：已使用 Python 3.9.25、NumPy 1.26.4 核对输出、数组形状、与原来合并表达式的等价性、重复调用及全局状态影响。Spyder 操作依据官方文档整理，编写时未实际操作 GUI 完整走一遍调试流程。

参考资料 (References)

1. <https://numpy.org/doc/stable/reference/random/generated/numpy.random.Generator.normal.html>
2. <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>
3. <https://docs.spyder-ide.org/current/videos/first-steps-with-spyder.html#customization>
4. <https://docs.spyder-ide.org/current/panes/debugging.html>
5. <https://docs.spyder-ide.org/current/panes/variableexplorer.html>
6. <https://docs.python.org/3/library/pdb.html#debugger-commands>
7. <https://numpy.org/doc/stable/reference/arrays.ndarray.html#array-attributes>
8. <https://numpy.org/doc/stable/user/basics.broadcasting.html>
9. <https://numpy.org/doc/stable/reference/random/generator.html>
10. <https://numpy.org/doc/stable/reference/random/legacy.html>
11. <https://numpy.org/doc/stable/reference/random/generated/numpy.random.seed.html>
12. <https://docs.spyder-ide.org/current/panes/editor.html#running-code>